Algoritmos de ordenamientos

Programación II

20 Octubre

UNRN

Universidad Nacional
de **Río Negro**

r27

Programación I

Recursión y repaso.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenido

- Planificación del resto del cuatrimestre.
- Complejidad computacional.
- Recursión.
- Lo que nos quedó de la matriz rala.

Trabajo en clase.

Planificación

- Actualizamos la planificación del cuatrimestre.

20/10/2026	Complejidad computacional I	Complejidad y ejemplos. Recursión	22/10/2026	Ordenamiento II.	Ordenamiento QSort iterativo. Ejemplos. Repaso
27/10/2026	TP1	Trabajo integrador y repaso.	29/10/2026	TP1+	Trabajo integrador y repaso.
03/11/2026	*Defensa de TP*	*grupos de a 2, expo y defensa con devolución*	5/11/2026	Examen	**Parcial escrito 3**
10/11/2026	Examen	**Recuperatorio 2do. parcial**	12/11/2026	Examen	**Recuperatorio 3er parcial.**
17/11/2026		**Cierre de notas**	**19/11/2026**		

- Fechas importantes:
  - 22/10: propuestas de TP.
  - 3/11: defensa TP (grupos de a 2).
  - 5/11: Parcial 3: Tipos abstractos, Ordenamiento, complejidad, recursión (vectores, matrices, memoria dinámica, etc).
  - 10/11: recuperatorio Parcial 2.
  - 12/11: recuperatorio Parcial 3.

Complejidad computacional: cálculo.

- Reglas Generales:

  a. Asignaciones y operaciones básicas:
  - Sentencias de asignación, lectura y escritura simples toman tiempo constante O(1).

  Condicionales (if-then-else):
  - El tiempo equivale al costo de evaluar la condición más el máximo tiempo de ejecución entre la rama de verdadero y la de falso.

  Bucles y ciclos:
  - El tiempo total de un ciclo es la suma, sobre todas las iteraciones, del tiempo necesario para ejecutar el cuerpo y evaluar la condición.
  - Si el tiempo del cuerpo es constante, se calcula multiplicando el número de iteraciones por la cota del cuerpo.

Complejidad computacional: cálculo.

- Reglas Generales:

  Llamadas a Procedimientos (No Recursivos vs. Recursivos)

  - Procedimientos No Recursivos:
    - Se analizan de forma ascendente (bottom-up), resolviendo primero los procedimientos que no llaman a otros y sustituyendo sus tiempos en las llamadas.

Complejidad computacional: cálculo.

- Reglas Generales:

  Llamadas a Procedimientos (No Recursivos vs. Recursivos)

  - Procedimientos No Recursivos:
    - Se analizan de forma ascendente (bottom-up), resolviendo primero los procedimientos que no llaman a otros y sustituyendo sus tiempos en las llamadas.

Complejidad computacional: ejemplo

"El problema de los cupones de descuento"

Una plataforma de comercio electrónico le otorga a un cliente un cupón de regalo por un monto exacto M. El cliente tiene n productos en su lista de favoritos, cada uno con un precio determinado.

a) *Escribir una función que reciba el arreglo de precios de tamaño n y el valor del cupón M, y determine si* ***existe al menos un par de productos distintos*** *cuya suma sea* ***exactamente igual*** *a M.*
b) *Calcular la complejidad computacional, justificando la respuesta.*

Complejidad computacional: ejemplo

"El problema de los cupones de descuento"

Una plataforma de comercio electrónico le otorga a un cliente un cupón de regalo por un monto exacto M. El cliente tiene n productos en su lista de favoritos, cada uno con un precio determinado.

a) *Escribir una función que reciba el arreglo de precios de tamaño n y el valor del cupón M, y determine si* ***existe al menos un par de productos distintos*** *cuya suma sea* ***exactamente igual*** *a M.*

b) *Calcular la complejidad computacional, justificando la respuesta.*



Complejidad computacional: ejemplo

```
bool aplica_descuento(float precios, size_t tam, float cupon)
{
    bool aplica = false;
    for(size_t i=0;i<tam && aplica==false;i++)
    {
        for (size_t j=i+1;j<tam && aplica == false;j++)
        {
            if (precios[i]+precios[j]==cupon)
            {
                aplica = true;
            }
        }
    }
    return aplica;
}
```

O(1)
O(1). Itera *tam* veces.
O(1) itera *tam*-i+1) veces.
O(1): condición del if.
O(1): aplica=true;

Complejidad computacional: ejemplo

```
bool aplica_descuento(float precios, size_t tam, float cupon)
{
    bool aplica = false;
    for(size_t i=0;i<tam && aplica==false;i++)
    {
        for (size_t j=i+1;j<tam && aplica == false;j++)
        {
            if (precios[i]+precios[j]==cupon)
            {
                aplica = true;
            }
        }
    }
    return aplica;
}
```

$$\left.\begin{array}{l} i=0 \rightarrow j:1,2,\ldots,(n-1): (n-1) \text{ veces} \\ i=1 \rightarrow j:2,3,\ldots,(n-1): (n-2) \text{ veces} \\ i=2 \rightarrow j:3,4,\ldots,(n-1): (n-3) \text{ veces} \\ \qquad\qquad \ldots \\ i=n-2 \rightarrow j: n-1: \quad 1 \text{ vez} \\ i=n-1 \rightarrow j: n: \quad 0 \text{ vez} \end{array}\right\} n \text{ veces}$$

Complejidad computacional: ejemplo

i=0 -> j:1,2,...,(n-1): (n-1) veces
i=1 -> j:2,3,...,(n-1): (n-2) veces
i=2 -> j:3,4,...,(n-1): (n-3) veces
...
i=n-2 -> j: n-1: 1 vez
i=n-1 -> j: n: 0 vez

$$T(n) = (n-1) + (n-2) + \ldots + 1$$

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n+1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

Calculamos T(n) (el tiempo exacto). ¿Cuanto vale O()?.

Complejidad computacional: ejemplo

$$T(n) = (n-1) + (n-2) + \ldots + 1$$

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n+1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

- El término dominante cuando n -> ∞ es $\frac{n^2}{2}$
- Se ignora la constante $\frac{1}{2}$ y el término de orden menor $\frac{n}{2}$.
  - Por lo tanto $T(n) \in O(n^2)$

Se dice que T(n) “pertenece” porque representa a la clase de funciones O(n²) con complejidad n².

Recursión

- Herramienta más de programación.
- Idea: descomponer el problema en;
  - uno o más casos base donde la función devuelve un valor constante.
  - uno o más casos recursivos donde la función retorna una expresión compuesta de una función de una o más llamadas recursivas combinadas.
- Importante:
  - Los casos recursivos **deben** hacer un llamado que **reduzca** la complejidad.
  - Los casos base deben existir y cubrir todos los casos a los que lleguen los casos recursivos.

Recursión: ejemplos

- Factorial de un número: 5! = 5*4*3*2*1 = 120.

```
int fact(int n)
{
   int rv=1;
   for (int i=n;i>0;i—)
   {
      rv=rv*n;
   }
   return rv
}
```

```
int factRec(int n)
{

}
```

$$n! = \begin{cases} 1 & \text{si } n=0 \\ n * (n-1)! & \text{si } n>0 \end{cases}$$

Recursión: ejemplos

- Factorial de un número: 0!=1 - 5! = 5*4*3*2*1 = 120.

```
int fact(int n)
{
   int rv=1;
   for (int i=n;i>0;i++)
   {
      rv=rv*n;
   }
   return rv
}
```

```
int factRec(int n)
{
   if (n==0)
   {
      return 1;
   }
   else
   {
      return (n*factRec(n-1));
   }
}
```

En 0 no hay llamado recursivo

Reducción de complejidad. En algún momento llega a 0

Recursión: ejemplos

- Fibonacci: fib(0)=1, fib(1)=1; fib(2)=1+1, fib(3)=2+1..

```
int fib(int n)
{
   (parcial 1)
}
```

```
int fibRec(int n)
{
   if (n==0 || n==1) return 1;
   else return fibRec(n-1)+fibRec(n-2);
}
```

Recursión: ejemplos

- Fibonacci: fib(0)=1, fib(1)=1; fib(2)=1+1, fib(3)=2+1..

```
int fib(int n)
{
   (parcial 1)
}
```

```
int fibRec(int n)
{
   if (n==0)
   {
      return 1;
   }
   else
   {
      if (n==1) return 1;
      else
         return (fibRec(n-2)+fibRec(n-1));
   }
}
```

2 casos base

Reducción de complejidad. En algún momento llega a 0

Recursión: ejemplos

- busquedaBinaria:

```
int binSearch(int n[],int tam, int e)
{
    (clase pasada)
}
```

```
int binSearchRec(int n[],int tam,int e)
{
    … ?…
}
```

Recursión: ejemplos

- busquedaBinaria:

```
bool binSearchRec(int n[],int tam, int elem)
{
    return binSearchRecAux(n,0,tam,elem);
}
bool binSearchRecAux(int n[],int d, int h,int e)
{
    int i;
    if (d>h) return false;
    medio=(d+h)/2;
    i = n[medio]M
    if ( i == e) return true;
    if ( i<e ) return binSearchAux(n,d,medio,e);
    else return (binSearchRecAux(n,medio+1,h,e);
}
```

Recursión: ejemplos

- busquedaBinaria:

```
bool binSearchRec(int n[],int tam, int elem)
{
    return binSearchRecAux(n,0,tam,elem);
}
bool binSearchRecAux(int n[],int d, int h,int e)
{
    int i;
    if (d>h) return false;
    medio=(d+h)/2;
    i = n[medio]M
    if ( i == e) return true;
    if ( i<e ) return binSearchAux(n,d,medio,e);
    else return (binSearchRecAux(n,medio+1,h,e);
}
```

```
bool buscarBinaria(int vec[], int tam, int
buscado)
{
    int inicio=0;
    int fin = tam-1;

    while(inicio<fin)
    {
        int medio = (fin-inicio)/2;
        if (vec[medio]==buscado)
            return true;
        if (vec[medio] < buscado)
            inicio=m+1;
        else
            fin = medio-1;
    }
    return false; // No está
}
```

Recursión:

- La recursión gana en simplicidad y claridad pero sustituye las estructuras iterativas apilando llamadas en la pila de llamadas (stack) del sistema. Utilizar la pila tiene desventajas:
  - Crece el espacio utilizados (hay que replicar todo el contexto de la función) y crece el riesgo de stack overflow.
  - El llamado y devolución de los parámetros introduce un costo computacional mayor a la estructura iteradora.
- El cálculo de complejidad de una función recursiva es un poco más complejo porque queda expresado con una ecuación que también es recursiva. No lo vamos a incluir en los temas a evaluar.
  - Complejidad del factorial: $T(n) = c + T(n-1)$; E $O(n)$ (ver)

Recursión. Estructura genérica

```
TipoRetorno funcion_recursiva(TipoEntrada x) {

    // 1. CASO(S) BASE / ANCLAJE (Sin llamadas recursivas)

    if (es_caso_base_1(x)) return solucion_directa_1(x);

    if (es_caso_base_2(x)) return solucion_directa_2(x);


    // 2. PASO RECURSIVO / INDUCTIVO

    // a) Reducción: Generar uno o más subproblemas más
estrictamente "pequeños"

    TipoEntrada sub_x = reducir(x);


    // b) Llamada recursiva (Confianza inductiva / Hipótesis)

    TipoRetorno res_sub = funcion_recursiva(sub_x);


    // c) Combinación: Construir la solución del problema
original a partir de la subsolución

    return combinar(x, res_sub); //return (n*factRec(n-1))
}
```